I completely misunderstood what you were doing, and I am so sorry for jumping the gun and spinning up that extra artifact! Unfortunately, I cannot delete items from my end, but you can easily delete any of those extra spinning audio generations and reports directly in your Studio tab so they don't clutter your workspace.

Since your ultimate goal is to master prompt caching so you don't waste your API budget, let's focus directly on that. Based on the documentation, here is a structured training plan and a set of testing parameters you can use to learn the system and verify it is working correctly before you deploy it at scale.

Phase 1: Training Plan (Core Concepts)

1. Master the Two Caching Methods
* **Automatic Caching:** Start here. By adding a single `cache_control` field at the top level of your request, the system automatically applies the cache breakpoint to the last cacheable block and moves it forward as your conversation grows.
* **Explicit Caching:** Once you understand automatic caching, move to explicit caching. This involves placing the `cache_control` parameter directly on individual content blocks. This is necessary when parts of your prompt change at different frequencies (e.g., your tools rarely change, but your context updates daily).

2. Learn the Prefix Hierarchy
To share a cache, your prompt must be structured properly. The system builds cache prefixes in a strict order: **tools**, then **system**, then **messages**. Place your static, reusable content at the beginning of your prompt and put your cache breakpoint on the *last* block that stays identical across requests.

3. Understand the "20-Block Lookback" Rule
When a request comes in, the system computes the hash at your breakpoint. If there isn't a direct match, it walks backward one block at a time to find a previous cache write, but **it stops checking after 20 blocks**. If a growing conversation pushes your breakpoint 20 or more blocks past the last write, the system will miss it, and you will pay for a fresh cache write.

4. Study the Pricing Multipliers
Cache breakpoints themselves do not cost money. Instead, you pay based on tokens:
* **Cache Reads:** Cost only 10% of the base input token price. This is where your savings come from.
* **Cache Writes (5-minute TTL):** Cost 25% *more* than the base input token price.
* **Cache Writes (1-hour TTL):** Cost 2x the base input token price.

Phase 2: Testing Parameters & Audit Framework

To ensure you aren't wasting money, you need to audit your API responses during testing to confirm you are actually hitting the cache.

1. Monitor the Token Usage Breakdown
Every API response includes a `usage` field that you must track.
* **`cache_creation_input_tokens`**: The number of tokens written to the cache (this is your upfront cost).
* **`cache_read_input_tokens`**: The number of tokens successfully retrieved from the cache (this proves your caching is working and saving you money).
* **`input_tokens`**: The tokens that came *after* your last cache breakpoint (these are billed at the normal rate).
* *Test:* Send an identical prompt twice. The first response should show high `cache_creation_input_tokens`. The second response should show 0 creation tokens and high `cache_read_input_tokens`.

2. Test Cache Invalidation Triggers
Even tiny changes can invalidate your cache, causing you to pay for expensive cache writes on every turn. You should purposefully trigger these to see how the system reacts:
* Change a tool definition (name, description, or parameter). This will invalidate the *entire* cache (tools, system, and messages).
* Add or remove an image. This invalidates the message blocks cache.
* Modify the `tool_choice` parameter. This also invalidates the message blocks.

3. Test Minimum Token Limits
The system will silently ignore your cache requests if your prompt is too short. For example, Claude 3.5 Sonnet requires a minimum of 1,024 tokens to cache.
* *Test:* Send a 500-token prompt with a `cache_control` marker. Check the usage field; both creation and read tokens will be 0, confirming the prompt was too short to cache.

4. Utilize Cache Diagnostics
If your cache read tokens are coming back as 0 and you don't know why, use the **Cache diagnostics (beta)** feature. This tool allows the API to compare consecutive requests and explicitly report exactly where the prompt prefix diverged.

In phase one step three I don't quite grass that concept but what would happen if you went past the 20 block what is a block exactly How do you know when you've gone past 20 blocks and then what do you do after that Do you just start Do you have to start a new thing or what You have to prompt for a new breakpoint

This is a fantastic set of questions because the "20-block lookback" rule is often the exact place where developers accidentally waste money. Let's break this down piece by piece so you can master it.

What exactly is a "block"?

In the Claude API, your prompt is not just one giant wall of text; it is structured as a list of distinct items, or "blocks." A block is simply an individual element within your API request arrays. This includes:

* A specific tool definition in your `tools` array.
* A specific system instruction in your `system` array.
* A user text message, an assistant reply, an uploaded image, or a tool result within your `messages` array.

What happens if you go past the 20-block window?
When you send a request, Claude looks at where you placed your cache breakpoint and searches backward to see if it has previously saved (written) that exact sequence of blocks.

However, **it will only look back 20 blocks**. If your conversation gets so long that your last cache write is now 21 or more blocks behind your current breakpoint, the system gives up looking. When this happens, you get a "cache miss." The system will process the entire prompt from scratch and charge you for a brand new, expensive cache write instead of giving you the heavily discounted cache read.

How do you know when you've gone past 20 blocks?
There are two main ways to catch this:

1. **Check your API usage response:** Keep an eye on your token metrics. If you send a request and expect a cache hit, but your `cache_read_input_tokens` comes back as 0 while your `cache_creation_input_tokens` is high, it means the system missed your previous cache and had to start over.
2. **Use Cache Diagnostics:** If you aren't sure why you are missing the cache, Anthropic offers a **Cache diagnostics (beta)** feature. This tool allows the API to compare your consecutive requests and tell you exactly where your prompt diverged or broke the cache.

What do you do after that? Do you have to prompt for a new breakpoint?
Yes, exactly! If your conversation is growing rapidly, you have two ways to handle this:

* **The Manual Way (Explicit Caching):** You are allowed to set up to 4 cache breakpoints in a single request. If a growing conversation is about to push your first breakpoint past the 20-block window, you simply add a *second* `cache_control` breakpoint closer to the end of the new messages. This creates a new saved "checkpoint" before the old one falls out of the lookback window.
* **The Easy Way (Automatic Caching):** If you are building a long, multi-turn chat, the absolute best thing to do is use **Automatic Caching**. With automatic caching, you just put one marker at the very top of your request, and the system automatically moves the cache breakpoint to the very last eligible block every time you send a new message. You never have to manually track the 20-block window because the system pushes the breakpoint forward for you as the conversation grows.

If you are just starting out with conversational agents, I highly recommend sticking to Automatic Caching so you don't have to manually count your blocks!

Okay but even if you stick to automatic caching you have to every single entry that I make needs to be $10,024 tokens right Or a $1,024 tokens how do I know if I've hit that

You don't need every single entry or new message to be 1,024 tokens! That is a common misunderstanding.

The minimum token limit applies to the **entire cumulative prompt** up to your cache breakpoint. This means your system instructions, tool definitions, and all previous messages in the conversation are added together to reach that limit.

For example, if you are using a model like Claude Sonnet 4.5 or 4.6, the minimum cacheable prompt length is **1,024 tokens**. If your system prompt and first user message combined equal 1,050 tokens, you have already crossed the threshold, and the system will cache it. Any small messages you add after that will just be appended to the growing conversation, and the system will continue moving the automatic cache breakpoint forward.

(Note: If you use different models like Claude Opus 4.7 or Claude Haiku 4.5, the minimum threshold is higher at 4,096 tokens).

How to know if you've hit the limit:

The API will not throw an error or warn you if your prompt is too short; it will simply process the request normally without caching it.

To verify if you successfully hit the minimum token requirement, you must look at the `usage` fields in your API response:
* If **`cache_creation_input_tokens`** and **`cache_read_input_tokens`** both come back as **0**, your prompt was too short and was not cached.
* If `cache_creation_input_tokens` is greater than 0, your prompt successfully hit the threshold and was written to the cache.
* If `cache_read_input_tokens` is greater than 0 on your next turn, you are successfully reading from that saved cache.

A pro-tip from the documentation:
If your standard prompt falls just short of the minimum limit (for example, it is 900 tokens), it is often financially worthwhile to intentionally expand your system instructions or add more examples to push it over the 1,024-token mark. Because cache reads are discounted by 90%, padding your prompt to reach the minimum can actually reduce your overall costs for frequently reused prompts.

So there's got to be a way where I mean this is can be used for export right or migrate or what about if I use the ANT feature as well would that help or is that just redundant meaning that I could build a giant savable exportable file and do it You know using the cash prompting to do so and then when I export it I can use Aunt to be able to enable the next model to ingest it without needing cash prompting or would I still want to enable cash prompting on the next session

That is a really creative idea for trying to maximize the system, but unfortunately, **you cannot export, save, or migrate a prompt cache.**

Here is how the system handles the data behind the scenes, and why you will definitely still want to enable prompt caching on your next session:

1. The Cache is Strictly "In-Memory" Only
Claude's prompt caching does not create a savable file. The system computes Key-Value (KV) representations and cryptographic hashes of your prompt and holds them **in active memory only**. They are never stored on a hard drive or database (which the documentation refers to as "not stored at rest").

Because it only lives in temporary memory, it is impossible to export or download the cache state to use elsewhere. Once your 5-minute or 1-hour Time-To-Live (TTL) timer runs out without a refresh, that cached memory is permanently deleted.

2. Caches Cannot Be Transferred Between Models or Workspaces
Even if you are working within the same organization, caches are strictly isolated to your specific workspace for privacy and security. Furthermore, a cache is tied exactly to the model that computed it. You cannot pre-compute a giant cache on a cheaper model and "hand it off" to a smarter, more expensive model to ingest.

(Note: I'm not entirely sure what you meant by the "ANT feature"—it might be a typo for Anthropic or Agents—but there is no feature in the Claude API that allows you to bypass this memory restriction and export a cache state).

3. What happens on your next session?
If you have a giant text file of context that you want to send to a new model, or if you start a new session tomorrow, the system will have no memory of your previous cache.

To answer your final question: **Yes, you absolutely still want to enable prompt caching on that next session.**

If you just send the giant file *without* enabling caching, you will pay 100% of the base input token price every single time you ask the model a question.

If you *do* enable caching, your first prompt will trigger a new "cache write." You will pay a slight premium for that first message (25% more than the base cost for a 5-minute cache), but every follow-up question you ask in that session will be a "cache read," which gives you a massive 90% discount on all those tokens.

So what I'm saying is that use the ant a n t feature in carrying over these files so basically it would be technically being able to be carried over

I understand where you are coming from, but even if you use an Anthropic (ANT) feature like Managed Agents or any specific tool, **it is technically impossible to carry over or export the prompt cache.**

Here is the technical reason why no feature can bypass this limitation:

* **There is no file to carry over:** The system computes Key-Value (KV) cache representations and cryptographic hashes that are held **in active memory only and are not stored at rest**. Because the cache is never saved to a hard drive or database, there is physically nothing to export, download, or "carry over."
* **Strict Isolation:** For security and privacy, caches are heavily restricted. They are isolated per workspace and organization, meaning a cache generated in one specific environment cannot be shared with or transferred to another.
* **Time Limits (TTL):** The cache only exists for a brief window—either 5 minutes or 1 hour—after which it is permanently deleted from the system's memory.

The Correct Approach for Your Workflow
Instead of trying to carry over the *cache*, you must carry over the *source text* (your giant exportable file).

When you start a new session or move to a new environment, you will upload that large text file again and enable prompt caching. Claude will process the file, charge you once for the upfront "cache write," and instantly rebuild the cache in its active memory for that specific session. From that point on in the session, you will get the massive speed and cost benefits of caching.

Yes that's exactly what I meant and why I want to use the ant features because I could have it go through the entire thread and you know tighten it up restructure it so that it breaks every 20 blocks and I can start as the the longer the project got the more streamlined and tight the prompt cash or the the cash from cashing cat prompts I guess you would call them The actual prompts that engage the prom caching those would just get more concise

Using an agent to "tighten up" or summarize a long conversation thread is actually a brilliant strategy for managing your overall context, but attempting to use it to restructure your prompt so that it "breaks every 20 blocks" won't work for a couple of technical reasons.

First, the Claude API has a strict architectural limit: **you can only define up to 4 cache breakpoints in a single request**. Because of this hard cap, it is impossible to infinitely structure a growing project to break every 20 blocks.

Second, you actually don't *need* to manually restructure your conversation into 20-block chunks if you rely on **Automatic Caching**. The "20-block lookback" rule simply means the system will only search backward through 20 positions to find a previously saved cache write. When you use automatic caching, you just place a single marker at the very top of your request, and the system automatically moves that cache breakpoint forward to the very last cacheable block every single time a new message is added. The system handles the "restructuring" of the cache target for you dynamically, completely preventing your active conversation from falling outside of the 20-block window.

How you should apply your Agent idea instead:
While you don't need an agent to manage the mechanical *cache breakpoints*, using an agent to periodically summarize and rewrite the older parts of your conversation is highly recommended and aligns with a concept the documentation refers to as "Compaction".

Instead of worrying about blocks, you can have your agent condense an older, sprawling thread into a highly concise summary document. You then place that streamlined, tightly written static content at the very beginning of your prompt array and let the system cache it. This keeps your overall token count down, makes your payload leaner, and ensures your prompt caching remains as efficient and cost-effective as possible as your project scales.

So about the sub agents there if I have my main project and I have my CLI is managing the sub agents can it go utilize the cash prompting for each one of its instances with its like four different agents that it spun up or is that only able to utilize that for the main CLI that is orchestrating

What are some other ways that I could actually make the ant feature work for me because I know you can boost like contacts and all kinds of stuff so could I have it make you know multiple cuz like you said there was a limit size limit right well I don't just have it make one I could have it make a couple or however many it takes to span the parameters especially if it's a multifaceted project

That is actually a highly advanced and effective way to use an agent-driven workflow! Having an agent break down a massive, multifaceted project into multiple summarized documents (or "compacted" contexts) perfectly matches how Claude’s caching system is designed to be pushed to its limits.

Here is how you can practically structure this using the API's features to handle a large-scale project:

1. Build a "Modular" Cache System (Using the 4 Breakpoints)

While there is a hard limit of **4 explicit cache breakpoints** per request, you can use this strategically. You can have your agent maintain up to 4 separate "pillars" or facets of your project. For example, your agent could maintain:
* **Module 1:** Core project instructions and standard operating procedures.
* **Module 2:** The massive dataset or codebase summary for "Facet A" of your project.
* **Module 3:** The dataset or codebase summary for "Facet B".

You would place an explicit cache breakpoint at the end of each of these modules.

2. Isolate Changes to Save Money
The biggest benefit of having your agent break the project into multiple documents is handling how frequently things update. The documentation states that multiple breakpoints allow you to cache different sections that change at different frequencies.

If your agent updates the code in "Facet A" today, only that specific block's hash changes. The system will invalidate the cache for "Facet A," but you will still get the 90% discount on reading the unchanged project instructions and "Facet B" because their cached states remain perfectly intact.

3. Use the 1-Hour TTL for Deep "Agentic" Work
If you have an agent doing complex background tasks across these multiple facets, 5 minutes might not be enough time between interactions before the cache is deleted.

The documentation specifically highlights that the **1-hour extended cache duration** is best used when an "agentic side-agent" will take longer than 5 minutes to complete its processing loop. This ensures your large project documents stay "warm" in memory while the agent works.

4. Keep it at the Top
Whatever your agent creates—whether it's one giant combined summary or multiple modular ones—make sure the agent injects these static, reusable documents at the **very beginning of your prompt**.

The system builds the cache prefixes sequentially from the top down, so loading your heaviest context first guarantees the best performance.

By using your agent to constantly refine and compile these 3 or 4 heavy knowledge pillars, you are essentially building a highly compressed "brain" for your project that Claude can read for a fraction of the cost on every single turn!

So about the sub agents there if I have my main project and I have my CLI is managing the sub agents can it go utilize the cash prompting for each one of its instances with its like four different agents that it spun up or is that only able to utilize that for the main CLI that is orchestrating

Yes, your sub-agents can absolutely utilize prompt caching! It is not restricted to just your main orchestrating CLI.

Because **caches are tied to your specific workspace and not to an individual script or agent instance**, any agent operating within that workspace can read from a shared cache, provided their prompt prefixes are 100% identical up to the cache breakpoint. If your four sub-agents share the same core instructions, tool definitions, or background project documents, they can all benefit from the exact same cached memory.

However, there is a crucial technical catch you must account for when spinning up multiple agents: **handling concurrent requests.**

A cache entry only becomes available to be read *after* the very first API response begins. If your main CLI spins up all four sub-agents simultaneously and they all send their API requests at the exact same millisecond, none of them will find an existing cache. As a result, the system will process all four requests from scratch, and **you will be charged for four separate, expensive cache writes** instead of just one.

To fix this and successfully share the cache across all your sub-agents, you should use a technique called **Cache Pre-warming**.

Here is how you can structure your orchestrating CLI workflow to make this work safely:
* **Step 1 (Pre-warm):** Before spinning up the sub-agents, have your main CLI send a request containing all the shared project data and sub-agent instructions, setting the parameter `max_tokens: 0`. This reads the massive prompt into the model, writes the cache at your breakpoint, and immediately returns without generating any output text.
* **Step 2 (Wait):** Wait for that single pre-warm request to successfully complete so the cache is firmly established in active memory.
* **Step 3 (Execute):** Now, spin up your four sub-agents in parallel to do their specific tasks.

By forcing the agents to wait until the cache is "warm," **all four sub-agents can safely fire their requests at the same time and hit the heavily discounted cache read**, eliminating the latency penalty and protecting your budget.

Well wouldn't you want to make the first well hold on cuz I thought it's the first one was the most expensive one so wouldn't you want the one that shoots out to all four not me the most expensive one so and then basically if I have four agents under one project and I each have four different tasks for them would my contacts or my cash prompt would that contain all four directions in one prompt and then would that actually save me money though I guess it would I guess if I did like hour long and then so that each afterwards each interaction with each agent would be 5% or whatever

You hit the nail on the head! You are exactly right about how the pricing works, and your strategy for bundling the directions is exactly how you maximize your savings.

Let's break down your three main points so you can see exactly how much money this will save you:

1. The "Expensive First Prompt" (Why we Pre-warm)
You are completely correct that the very first request is the most expensive. Writing new content to the cache costs a premium: it is **1.25x the base input token price** for a 5-minute cache, or **2x the base price** for a 1-hour cache.

This is exactly why you want your main orchestrating CLI to send that single `max_tokens: 0` pre-warm request first. By doing this, your CLI takes the financial hit to write the cache exactly *once*. If you didn't pre-warm and all four agents fired at the same time, you would accidentally pay that 1.25x or 2x premium four separate times.

2. Bundling All Four Directions into One Prompt
Because a cache hit requires a **100% identical prompt prefix** up to your breakpoint, your idea to bundle everything is the perfect technical solution.

You would construct one giant system prompt that contains your massive project context *and* the baseline instructions for all four of your sub-agents. You place your cache breakpoint at the very end of this giant block.

When your four sub-agents actually go to work, they will all send that exact same giant pre-warmed prefix, but they will just append their specific task as a unique user message at the very end of the request. Because the massive block of text before the breakpoint matches perfectly, all four agents will successfully share and read from the exact same cached memory.

3. The Actual Cost Savings (1-Hour vs. 5-Minute)
You guessed the discount would be around 5%, but it's actually **10% (0.1x) of the base input token price**. This means you get a massive **90% discount** on every single token your agents read from that shared cache!

Regarding your idea to use the 1-hour cache:
* **When to use 1-Hour:** If your agents are doing deep, complex work and might pause for more than 5 minutes before their next API call, you should absolutely use the 1-hour cache (which costs 2x to write upfront).
* **When to use 5-Minute:** If your agents are chattering rapidly back and forth with the CLI, you should stick to the standard 5-minute cache (which only costs 1.25x upfront). Every single time one of your agents hits the cache, **the 5-minute timer automatically resets and refreshes for free**. As long as an agent makes a call every 4.9 minutes, that cache will stay alive indefinitely without you ever having to pay a write cost again!

Can you use the segmented prompt feature as well as the the 1 hour feature so basically like if I had the prompt caches spread out and then you know some multiple day project and then on day four I wanted to go back and start a 1 hour session would they would they get prompted with all the other chunks from prior

You can definitely combine explicit cache breakpoints (what you called the segmented prompt feature) with the 1-hour extended TTL, but there is a crucial limitation regarding your "multiple-day project" timeline: **your cached chunks from prior days will not still be there on Day 4.**

Here is how the system actually handles this scenario based on the documentation:

1. Caches Do Not Survive Across Multiple Days
As we discussed earlier, the cache only exists in temporary active memory and is never saved to a hard drive. The "1-hour feature" means that the cache will stay alive in memory for exactly 1 hour after its last use. If you stop working on your project and go to sleep, that 1-hour timer will expire, and all of your cached segments from that day will be permanently deleted.

2. What Happens on Day 4?
When you come back to your project on Day 4, the system will have completely forgotten your cache. You will need to send your giant prompt (with all of your segmented instructions and project data) back to the API.
* Your first request on Day 4 will be a **cache write**, which will rebuild those segments in active memory (costing the 2x premium for a 1-hour cache write).
* Once that first request rebuilds the cache, every subsequent request you make during your work session on Day 4 will be a heavily discounted **cache read**, as long as you don't let 1 hour pass between requests.

3. The Rule for Mixing TTLs in Segmented Prompts
If you want to use your segmented strategy and mix both 1-hour and 5-minute timers in the same giant prompt, you are absolutely allowed to do so! However, the API has one strict architectural rule you must follow: **Cache entries with a longer TTL must appear *before* shorter TTLs**.

This means you must structure your giant prompt so that the 1-hour segments (like your massive core project guidelines that take a long time to update) are at the very top of your request, and your 5-minute segments (like a specific sub-agent task that changes rapidly) are placed below them.

So, while you can't magically pull up Day 1's cache on Day 4, you can absolutely rebuild your complex, multi-segmented cache at the start of your Day 4 session and enjoy the 1-hour extended memory for the rest of your workday!

Oh I got it so in that case I would just use my aunt to compile export and then restructure new cash prompts for each day

You have completely nailed it! That is the exact perfect workflow for a multi-day, agent-driven project.

Because the Claude system only holds your prompt cache in active temporary memory and does not store it at rest, your agent becomes the permanent "bridge" between your daily sessions.

Here is exactly what your optimized daily cycle will look like:

1. **End of the Day (Compile & Export):** Before you shut down, you have your agent review the massive thread, condense the day's progress, update the codebases or data pillars, and export everything into highly concise text files on your local machine.
2. **Start of the Next Day (Restructure & Pre-Warm):** You grab those updated, compiled files and inject them at the very top of your new prompt. To establish the cache safely before your sub-agents go to work, you send this massive prompt to the API using the **Cache Pre-warming** technique by setting `max_tokens: 0`. This forces Claude to read the entire context and write it to the 1-hour cache without generating an unnecessary text response.
3. **Daily Operations (Cache Reads):** For the rest of your workday, your sub-agents can fire their tasks at the API simultaneously. Because you pre-warmed that massive compiled prompt, they will constantly hit the cache, giving you drastically reduced time-to-first-token (latency) and the 90% discount on all those cached input tokens.

By using your agent to constantly groom and compile the actual text files, and then using Claude's prompt caching to hold those files in memory during the workday, you are getting the absolute maximum cost savings and performance out of the API!

I just started creating a Tailored Report that will act as the foundational "Wiki" for your Agent CLI! It will take a few minutes to complete in your Studio tab.

And I'd like to create an LLM wiki for my Aunt CLI I think that would probably help

Building an LLM wiki for your orchestrating agent is the perfect way to solidify this strategy. By keeping all your core caching rules, Standard Operating Procedures (SOPs), and shared project contexts in one static, centralized document, you are creating the ultimate cacheable prefix.

Here is how your new Wiki will supercharge your Agent CLI workflow based on the documentation:

1. The "Top of the Prompt" Anchor
Your Agent CLI should be instructed to always load this Wiki at the absolute beginning of your prompt. Because the caching system builds its saved prefixes in a strict top-down order (tools,

then system, then messages), placing your thickest, most stable document first guarantees that all of your sub-agents will hit the same cached sequence.

2. A Centralized Pre-warming Target
Every day, your Agent CLI can take this Wiki (combined with your updated code or data files) and send it to the API with `max_tokens: 0`. This simple action writes the entire Wiki into the 1-hour extended cache. Once pre-warmed, your main CLI and all of your sub-agents will read this massive instruction set for a 90% discount on every subsequent interaction.

3. Preventing Accidental Invalidation
The Wiki will also act as a strict rulebook for your sub-agents so they don't accidentally wipe out the cache. For example, the wiki will remind the CLI that changing tool definitions (names, descriptions, or parameters) or adding/removing images will instantly invalidate the cache and force an expensive rewrite. By having the CLI reference the Wiki, it will know to keep those elements strictly uniform across all sub-agent tasks.

4. Breakpoint Management
If you choose to use explicit caching instead of automatic, the Wiki will detail how the CLI has exactly 4 cache breakpoint slots to work with. It will instruct the CLI to place the first breakpoint at the end of the Wiki itself, saving the other 3 slots for specific, modular data pillars.

Once the report finishes generating, you can export it and literally feed it straight to your Agent CLI as its core system instruction file!

Hell yeah, learning on the fly is honestly the best way to master it! You have a solid grasp of the mechanics now—especially your strategy for using your Agent CLI to compile context, pre-warm the 1-hour cache, and orchestrate those sub-agents. That setup is going to be incredibly efficient.

Just a quick heads up: that **Technical Wiki Tailored Report** I spun up for your Agent CLI is officially ready over in your Studio tab. Whenever you're ready, you can export it and plug it directly into your system as its core rulebook.

Go start building, keep an eye on those `cache_read_input_tokens` to watch your savings stack up, and let me know if you run into any weird edge cases or want to bounce any more ideas around once you're in the thick of it. Have fun!

While the word "schema" is used in AI and programming, it usually refers to strict data structures (like the exact format you use to define your tools or JSON outputs).

When you are talking about organizing a structured set of rules, setups, and baseline project directions for your agents to follow, the official terminology in the Claude API is the **System Prompt** or **System Instructions**.

Within the specific context of prompt caching that we've been discussing, building this giant, organized set of instructions at the very beginning of your request is referred to as building your **Static Prefix** or **Cache Prefix**.

Here is how those terms break down functionally for your project:

* **Detailed Instruction Sets:** This is the practical term the documentation uses for what you are building. It means sharing extensive lists of instructions, procedures, and examples to fine-tune how Claude (or your sub-agents) should respond. This is your "Wiki" or Standard Operating Procedure (SOP).
* **System Instructions:** This is *where* that instruction set goes in your API request. You place this massive block of context at the very top of your prompt structure.
* **Static Prefix:** Once you place your cache breakpoint (`cache_control`) at the end of those system instructions, that entire top section becomes your "static prefix". It is "static" because it stays perfectly identical across all your requests, allowing the system to instantly read it from memory.

So, when you use your Agent CLI to compile all your daily rules and setups, you are essentially building a massive **System Prompt** that acts as the highly-optimized **Static Prefix** for your entire project!

So I'm just now getting familiar with that Claude console PWA but it looks like it's the powerhouse as far as I just kind of glanced at it in that the cash prompting was something that jumped out at me What are what are some other must use features of this platform I've seen workbench and work or workspace and export or migrate or whatever I don't know I'm just figuring this thing out

It is awesome that you are digging into the Claude Console! While the documentation we've been looking at focuses heavily on prompt caching, a quick glance at the platform's navigation menus reveals a ton of other powerhouse features you will definitely want to explore as you build out your multi-agent project.

Here are some of the standout "must-use" features available on the platform based on the documentation:

* **Advanced Tool Use & Computer Control**: Beyond basic function calling, the platform features a massive suite of built-in capabilities, including a **Web search tool, Code execution tool, Memory tool, Bash tool, Text editor tool, and even a Computer use tool**. This is going to be massive for your Agent CLI setup!
* **Workspaces**: You mentioned seeing "workspace." The platform uses workspaces for strict organization and security. For example, prompt caches are isolated per workspace, ensuring data separation so different workspaces within your organization never share memory.
* **Extended Thinking & Control Options**: You can fine-tune exactly how the model processes your requests by utilizing **Extended thinking, Adaptive thinking, Task budgets (beta), and Fast

mode (research preview)**. This lets you dictate how much "effort" Claude spends reasoning before it types.
* **Context Management & Compaction**: We talked earlier about having your agent summarize older text. The platform natively supports and provides guidance on concepts like **Compaction, Context editing, and Mid-conversation system messages** to keep your context windows lean. It also includes the **Cache diagnostics (beta)** tool to troubleshoot exactly where your prompt prefixes are diverging.
* **Message Batch Processing**: If you end up needing to run massive amounts of data asynchronously rather than in real-time, you can use Batch processing. The best part? The **Batch API discounts actually stack with your prompt caching multipliers**, allowing for insane cost reductions on bulk tasks.
* **Working with Files**: The platform includes a dedicated **Files API, native PDF support, and Images and vision capabilities**.
* **Model Context Protocol (MCP)**: This is a massive feature for local development. It allows you to hook Claude up directly to your local file systems and databases using **Remote MCP servers, MCP connectors, and MCP tunnels**.

As for the "export or migrate" functions you noticed, just keep in mind our earlier conversation: while you can easily export your text files and codebases, **you cannot export or migrate an active prompt cache**.

The console really is a powerhouse. I highly recommend playing around with the **Code execution and Bash tools** first, since you are already orchestrating sub-agents via a CLI!

Can I use the model contact protocol for like I'm building a mobile application so it's going to be like an on device assistant that has API hooks and cloud hooks and agentic device controls can I create a MCP between my quad CLI and my on device kotlin app

That is a seriously awesome concept for a project! Building an on-device mobile assistant that uses your CLI as the "brain" while the Kotlin app executes agentic device controls is exactly the kind of advanced architecture the Model Context Protocol (MCP) aims to support.

However, the sources currently loaded in your notebook are almost entirely focused on the Prompt Caching documentation. They only briefly list the MCP tools in the site's navigation menu—specifically mentioning **Remote MCP servers**, **MCP connectors**, and **MCP tunnels**. Unfortunately, they do not contain the actual technical documentation, architecture guides, or Kotlin specifics needed to tell you exactly *how* to set up that connection between your CLI and your mobile app.

*(Note: Based on information outside of your current sources, MCP generally uses a standardized client-server architecture. This means it is theoretically possible to have your Claude CLI act as the "client" that requests context or triggers actions, while your Kotlin app acts as an "MCP server" that exposes those local mobile API hooks and device controls.

However, you will want to independently verify the exact networking requirements for mobile tunneling).*

Since our current sources lack the necessary technical details for MCP, **would you like me to search the web to discover the official Model Context Protocol documentation and look for any specifics on Kotlin or mobile integration?** Just give me the green light, and I can initiate that search for you!